

DEVELOPMENT_PLAN

Helios — Development Plan

Companion to: `DEPENDENCY_MAP.md` (the *what depends on what*) and Bible §12 (Claude Code workflows) + §23 (roadmap). · **Version:** v1.0 · **Date:** 2026-06-03

Purpose. This is the *how-we-build-it* plan: the milestones M0–M12 turned into concrete work packages, each with the Claude Code workflow to use, its definition-of-done, its exit gate, and what can run in parallel. It is written for **continuity and resume-ability** — a future session reads §2 (current status) + §12 (milestone tracker), finds the next work package, opens its spec doc, and starts.

Status legend: ✔ done · 🔄 in progress · todo. Update the tracker (§12) as packages land — it is the single resume point.

1. How to use this plan

1. Open §12 **Milestone Tracker**; find the lowest-numbered milestone not ✔.
2. Open that milestone's **work-package table** (§7–§9) for the task list, the spec doc, the workflow, and the DoD.
3. Build in dependency-safe order (never start a package before its deps are green — the tracker enforces this).
4. Run the package's tests; meet its **exit gate**; mark it ✔; update `CLAUDE.md` §10 (current status).
5. Keep the three continuity docs in lockstep (`CLAUDE.md`, `DEPENDENCY_MAP`, this plan).

2. Current status (2026-06-03)

Specification & continuity layer — DONE. The project is fully specified and the governance keystone is authored & validated:

Artifact	State
<code>docs/architecture/HELIOS_PROJECT_BIBLE.md</code> (25 sections)	✔
<code>docs/planning/DEPENDENCY_MAP.md</code>	✔
<code>CLAUDE.md</code>	✔
<code>models/semantic/semantic_layer.yaml</code> (A5.1, 47 metrics / 19 dims)	✔ authored + referential-integrity validated (0 dangling refs)
<code>docs/architecture/MCP_ARCHITECTURE.md</code> (A6.x spec)	✔
<code>docs/architecture/AGENT_ARCHITECTURE.md</code> (A8.x/A9.x spec)	✔

Code layer — not started. No repo scaffold, no GCP/dbt setup, no models, no servers, no agents, no eval. The semantic registry is *authored* but cannot be *compiled/served* until the marts it references exist (M3/M4).

Next action: M0 — repo scaffold + GCP/IAM/ADC + dbt config. (§7, WP-0.1.)

3. Guiding execution principles

1. **Governed-first.** Build the data spine and the semantic layer before agents. No code path hand-authors SQL or computes stats (G1–G5).
2. **TDD on SQL.** Write the dbt/golden test *before* the model. keystones (sessionization, funnel monotonicity, revenue reconciliation, `decompose_change`) get golden-value tests because they fail *silently*.
3. **Eval-driven dev (from M10).** Once the benchmark exists, no change merges that drops top-1 accuracy >2 pts or introduces any hallucination.
4. **Front-load keystones.** `int_ga4__sessionized` (A3.1), `int_ga4__funnel_steps` (A3.2), the registry (A5.1, done), and `stats-mcp.decompose_change` (A6.3) have the largest blast radius — build them carefully and early.

- 5. **Parallelize independent tracks** (§10): the data spine, the data-independent math (stats / experiment), and memory/finance can progress concurrently via subagents.
- 6. **Never start a milestone before its deps are green** (dbt build + tests; from M7, the loop runs; from M10, the eval gate).
- 7. **Docs in lockstep**. Adding/renaming an artifact updates DEPENDENCY_MAP + this tracker; a canonical-name change updates the Bible Reference Card + CLAUDE.md.

4. The Claude Code workflow per activity (Bible §12, made concrete)

Activity	How to use Claude Code
Planning	Use plan mode to expand each milestone into a step plan <i>checked against</i> DEPENDENCY_MAP.md before editing. This file is the tracker.
Architecture	The specs exist (Bible, MCP_ARCHITECTURE, AGENT_ARCHITECTURE). For any <i>new</i> component, write/extend a spec doc first, then code to it.
Coding	Generate dbt models from the registry YAML; use subagents to build independent tracks in parallel (§10); reference the exact spec doc in each prompt.
Testing	TDD: write the dbt test / golden test first, then make it pass. Keystones get golden-value tests (§8).
MCP development	Scaffold each server from MCP_ARCHITECTURE.md §6/§9 ; dogfood by registering the server and having Claude Code call its own tools; write contract tests (§11 of the MCP doc).
Documentation	Auto-generate dbt docs ; keep CLAUDE.md/DEPENDENCY_MAP/this plan current; draft the case-study writeup at L3 (§9).
Deployment	Claude Code writes the GitHub Actions CI (dbt build+test+eval) and the Cloud Scheduler entrypoint for autonomous runs (M10/M11).

4.1 Custom slash commands to create early (M0–M1)

Investing in these once pays off every package after:

- /resume — read the tracker + CLAUDE.md, report current state and the next package.
- /new-metric <name> — add a metric to semantic_layer.yaml + its test + bump version + re-run the registry validator.
- /new-model <layer> <name> — scaffold a dbt model + its schema.yml test stub.
- /new-mcp-tool <server> <tool> — scaffold a tool (signature, schema, error modes, contract test) per the MCP doc.
- /new-scenario <id> — add an eval scenario YAML + register it (M10).
- /run-eval [--smoke|—full] — run the harness, render the results table, compare to baseline.

5. Indicative schedule (solo portfolio cadence; relative, not committed dates)

Effort sizes are relative (S ≈ ½–1 day, M ≈ 1–3 days, L ≈ 3–6 days, XL ≈ 1–2 weeks). Anchored to Bible §8 sizing (L1 ≈ 1–2 wks, L2 ≈ 3–5 wks, L3 ≈ 6–10 wks total).

Phase	Maturity	Milestones	Indicative	Headline outcome
P0	L1 (Intern MVP)	M0 → M7	~Sprint 1–2 (≈2 wks)	One anomaly → governed Decision Brief in <5 min, 0 hallucinated columns
P1	L2 (Strong portfolio)	M8 → M10	~Sprint 3–6 (≈4–6 wks)	7-agent loop + eval harness: ≥85% root-cause vs ≤45% baseline in CI
P2	L3 (Top-1% undergrad)	M11	~Sprint 7–9 (≈3–4 wks)	Autonomous scheduled runs, forecasting/cohorts/RFM, full Critic battery, memory-driven learning
P3/P4	Production / Frontier	M12+	beyond	Multi-tenant, warehouse-agnostic, causal inference, auto-executed experiments

6. Parallelization & critical path

Three tracks run concurrently after **M0** (DEPENDENCY_MAP §7); use subagents:

- **Track A — Data spine (critical path):** M1 → M2 → **M3 (keystone)** → M4 core → M5 (registry live) → M6 (semantic + warehouse MCP).
- **Track B — Math & experiment (data-independent, start day 1):** stats-mcp (incl. `decompose_change`), experiment-mcp, report-mcp core — each with unit/golden tests. Joins at M7/M9.
- **Track C — Memory & finance (independent):** helios_memory DDL + seeds, finance facts, `dim_date / dim_items`. Gates report-mcp memory tools (A7.4) and the full loop.

All three converge at **M7/M9 (agents)**. The schedule-determining chain is Track A → agents → eval; the **co-critical** memory branch (A7.1→A7.4→Critic/Narrator) must converge before the full eval run (DEPENDENCY_MAP §4).

7. Phase 0 — L1 MVP (M0 → M7)

Goal: prove the governed spine end-to-end on one diagnosis path. **Exit (L1):** `reconcile('revenue', 'day')` matches a hand-written control query to the cent; 0 hallucinated columns; one anomaly → brief in <5 min.

WP	Milestone	Artifacts	Spec doc	Workflow	Definition of Done / tests	Size
WP-0.1	M0	A0.1, A0.3, A0.4, A0.5	Bible §16.7–16.8, §17	plan mode + scaffold	<code>dbt debug</code> connects; ADC authenticates; <code>mcp_servers.yaml</code> stub present	M
WP-0.2	M0	A0.2 CLAUDE.md	—	—	✅ done	—
WP-1.1	M1	A1.1–A1.6 macros/source/seed	Bible §16.3, §16.6	TDD	macros compile; <code>get_event_param / sessionize / channel_group_case</code> unit-tested; seed loads	M
WP-2.1	M2	A2.1–A2.3 staging	Bible §16.9, §8.1	TDD	<code>stg_ga4__events / stg_ga4__event_params</code> build as views; <code>not_null/unique</code> key tests pass	M
WP-3.1	M3 ★	A3.1 <code>int_ga4__sessionized</code>	Bible §8.5	TDD + golden	<code>session_key</code> unique; session-scoped source/medium derivation golden test (traffic_source fallback); <code>engaged_session</code> predicate	L
WP-3.2	M3 ★	A3.2 <code>int_ga4__funnel_steps</code>	Bible §10; CLAUDE.md §5	TDD + golden	<code>reached_*</code> max-downstream; monotonicity golden test (<code>sessions ≥ reached_view_item ≥ ... ≥ reached_purchase</code>)	M
WP-4.1	M4	A4.1–A4.7 core marts	Bible §8.3–8.4	TDD	<code>fct_funnel / fct_daily_funnel / dims</code> build; revenue_reconciles to the cent ; channel <code>accepted_values</code> (10 groups)	L
WP-4.2	M4 II	A4.8–A4.9 finance, A4.12 tests	Bible §8.3, §16.4	TDD	<code>fct_orders / fct_order_items</code> ; dedup by <code>transaction_id</code> ; net = gross – refund	M
WP-5.1	M5	A5.1 ✅ + A5.2 compile gate	Bible §14; SEMANTIC_LAYER.yaml	TDD	registry compiles against the real marts ; referential-integrity check wired as a test; worked example (§14.7) golden SQL	S
WP-6.1	M6 ★	A6.0, A6.2 semantic-mcp, A6.1 warehouse-mcp	MCP_ARCHITECTURE.md §6.1–6.2, §9	MCP dev + dogfood	<code>build_query → dry_run → run_query → reconcile</code> round-trips; byte-budget gate fires; unknown-name → hard fail; contract tests	L
WP-6b.1	M6b II	A6.3 stats-mcp	MCP_ARCHITECTURE.md §6.3, §9; AGENT_ARCHITECTURE.md §6.4	TDD + golden	decompose_change golden test (mix=−0.0018, rate=0, int=0); significance vs scipy ref; determinism (seed)	L
WP-6b.2	M6b II	A6.4 experiment-mcp, A6.5 report-mcp core	MCP_ARCHITECTURE.md §6.4–6.5	TDD	<code>power_analysis</code> vs closed-form ref; <code>render_brief([])</code> → EmptyFindings; design rejects ungoverned metrics	M
WP-7.1	M7	A8.0 framework, A8.1 Monitor, A8.6 Narrator(min), A9.1 runner, A12.1 basic brief	AGENT_ARCHITECTURE.md §4, §6.2, §6.7, §13	plan mode + subagents	L1 gate: one anomaly → brief <5 min; 0 hallucinated columns; audit log proves every SQL came from semantic-mcp	L

8. Phase 1 — L2 Strong Portfolio (M8 → M10)

Goal: the full 7-agent plan-execute-critique loop + the decomposition core, statistically defended and **benchmarked**. **Exit (L2):** ≥85% root-cause accuracy vs ≤45% baseline; 100% of findings carry significance + dollar impact; cost under byte budget.

WP	Milestone	Artifacts	Spec doc	Workflow	Definition of Done / tests	Size
WP-8.1	M8	A7.1 memory DDL, A7.2 vector store, A7.3 seeds, A7.4 report memory tools	Bible §22; MCP_ARCHITECTURE.md §6.5	TDD	save_diagnosis → recall_prior round-trips (BigQuery + ANN); seasonality/launch calendars seeded for the dataset window	L
WP-9.1	M9	A8.2 Decompose, A8.3 Diagnose, A8.4 Critic, A8.5 Prescribe, A8.7 Orchestrator, A9.3 audit, A4.13 exposures	AGENT_ARCHITECTURE.md §5–§10	plan mode + subagents	FSM routes correctly (clean-run short-circuit; DOWNGRADE→re-query≤MAX; DROP→suppression); a real run yields PASS findings w/ significance + \$ + experiment; hypothesis-tree pruning honored	XL
WP-10.1	M10 ★	A10.1–A10.7 eval harness, A11.1 CI	Bible §20; AGENT_ARCHITECTURE.md §12	eval-driven + deploy	injector + 50 labeled scenarios; scorers (rootcause/decomp/detection/dollars/hallucination/faithfulness); CI gate green: top-1 ≥0.85, hallucination =0, cost ≤ budget ; results table in PR comment	XL

9. Phase 2 — L3 Top-1% Undergrad (M11) and beyond

Goal: autonomy, statistical depth, adversarial rigor that read as production thinking. **Exit (L3):** <5 min/run sustained on schedule; eval accuracy holds across all canonical dimensions; memory-driven learning demonstrable.

WP	Milestone	Artifacts	Spec	DoD / tests	Size
WP-11.1	M11	A9.2 scheduler; forecast / cohort_retention / rfm_segment wired into Diagnose; A4.11 cohorts; full Critic battery; A12.2 export, A12.3 drill-down	Bible §23.3, §22; AGENT §6.5	scheduled autonomous runs land briefs; forecast-residual anomaly detection; cohort/RFM segments feed the hypothesis tree; Critic catch-rate measured on injected adversarial cases	XL
WP-11.2	M11	Case study / writeup	Bible §24–§25	published case study: architecture + the 85%-vs-45% benchmark + 3 walked-through diagnoses	M
WP-12+	M12+	Productionization (multi-tenant, warehouse-agnostic adapters, streaming) → Frontier (causal inference, auto-executed experiments)	Bible §23.4–23.5	per-phase exit criteria (e.g. two warehouses pass identical reconcile tests; a causal estimate validated vs a held-out experiment)	—

10. Testing & CI strategy across phases

- **Pyramid:** schema tests (not_null/unique/accepted_values/relationships) → data tests (revenue_reconciles , funnel monotonicity, rate bounds) → unit/golden tests (sessionization, decompose_change , build_query SQL snapshot) → contract tests (MCP tools) → integration (the eval harness).

- **Keystone golden tests (write these first):** session-key uniqueness; source/medium derivation; funnel monotonicity; revenue-reconciles-to-the-cent; `decompose_change` identity + the \$6.2 worked example; `build_query` \$14.7 SQL snapshot; the hallucination AST check.
- **CI turns on at M5** (registry compile) and becomes the **regression firewall at M10** (eval gate). On every PR touching `models/`, `semantic/`, `agents/`, `eval/`: `dbt build` + tests + eval (12-scenario smoke on push, full 50 on PR to `main`). Gates: top-1 ≥ 0.85 , decomp MAPE ≤ 0.10 , hallucination = 0 (hard), F1 ≥ 0.85 , dollar error ≤ 0.15 , faithfulness ≥ 0.95 , regression ≤ 2 pts.

11. Definition of Done per maturity level

- **L1 (M7):** governed spine runs end-to-end on one funnel; `reconcile` to the cent; 0 hallucinated columns; a single anomaly $\rightarrow$ a brief in <5 min.
- **L2 (M10):** seven agents + five MCP servers; Critic active; CI eval harness; significance + dollar impact on **every** finding; $\geq 85\%$ vs $\leq 45\%$ on the labeled benchmark; cost under byte budget.
- **L3 (M11):** autonomous scheduled operation with memory-driven learning; forecasting/cohorts/RFM; full Critic refutation battery; accuracy holds across all canonical dimensions; published case study.

12. Milestone tracker (the living resume point — update as packages land)

Milestone	Status	Key artifacts	Exit gate
Specs/docs	✓	Bible, <code>DEPENDENCY_MAP</code> , <code>CLAUDE.md</code> , semantic registry (A5.1), <code>MCP_ARCHITECTURE</code> , <code>AGENT_ARCHITECTURE</code>	all authored; registry validated (0 dangling refs)
M0 Foundation	← NEXT	A0.1, A0.3, A0.4, A0.5	<code>dbt debug</code> connects; ADC auth
M1 Sources & macros		A1.1–A1.6	macros compile; seed loads
M2 Staging		A2.1–A2.3	staging tests pass
M3 Sessionization + funnel ★		A3.1, A3.2, A3.3	monotonicity test passes; <code>session_key</code> unique
M4 Marts		A4.1–A4.9, A4.12, A4.14	revenue reconciles to the cent; 10-channel <code>accepted_values</code>
M5 Registry live	📦	A5.1 ✓ / A5.2	registry compiles against real marts; CI compile gate
M6 Grounding MCP pair		A6.0, A6.2, A6.1	<code>build_query</code> $\rightarrow$ <code>dry_run</code> $\rightarrow$ <code>run_query</code> $\rightarrow$ <code>reconcile</code> round-trips
M6b Stats/exp/report MCP II		A6.3, A6.4, A6.5	<code>decompose_change</code> golden test passes
M7 Minimal loop (L1)		A8.0, A8.1, A8.6, A9.1, A12.1	L1 DONE: anomaly $\rightarrow$ brief <5 min; 0 hallucinations
M8 Memory + report tools		A7.1–A7.4	save/recall round-trips; seeds loaded
M9 Full agent loop		A8.2–A8.5, A8.7, A9.3, A4.13	PASS findings carry significance + \$ + experiment
M10 Eval + CI (L2)		A10.1–A10.7, A11.1	L2 DONE: $\geq 85\%$ vs $\leq 45\%$; hallucination 0; cost $\leq$ budget
M11 Autonomy & depth (L3)		A9.2, A4.11, A12.2, A12.3, case study	L3 DONE: <5 min/run on schedule; accuracy holds all dims
M12+ Prod / Frontier		per Bible §23.4–23.5	per-phase criteria

13. Risk register

Risk	Likelihood	Impact	Mitigation
Sessionization / funnel flags silently wrong	Med	High	golden tests first (WP-3.1/3.2); monotonicity + reconcile gates before any agent consumes them
LLM agent produces plausible-but-wrong diagnoses	Med	High	grounding (G1–G5), deterministic stats, adversarial Critic, and the eval gate (≥85%) — the whole architecture is the mitigation
BigQuery cost overrun	Low	Med	mandatory <code>dry_run</code> ; <code>maximum_bytes_billed</code> cap at 5 GiB; partition pruning; smoke vs full eval split
Registry ↔ marts drift	Med	High	CI referential-integrity compile (A5.2); registry filename pinned; one physical file (CLAUDE.md guardrail)
Scope creep into dashboard/chatbot	Med	Med	anti-product stance in CLAUDE.md; Decision Brief is the only surface; conversation is secondary and late (M11)
Dataset limits (3-mo window, cookie identity)	Certain	Low	explicitly deferred items (Bible §23.7); LTV/cross-device/causal to P4; document caveats; Critic flags cookie-based user metrics
Solo continuity loss between sessions	Med	Med	the three lockstep docs + <code>/resume</code> command + this tracker as the single resume point

14. Cadence (running it solo)

- **Per session:** `/resume` → pick the next  package → plan-mode the steps against `DEPENDENCY_MAP` → TDD it → meet the exit gate → mark  → update CLAUDE.md §10.
- **Per milestone:** demo the headline outcome (even to yourself); keep a short changelog; ensure CI is green before advancing.
- **At L1/L2/L3:** record the demo + numbers for the case study and resume bullets (Bible §24–§25) while they're fresh.